

```
.aggregationStrategy (new
ResponseAggregator())
.parallelProcessing()
.executorService (executorService)
.to ("subRoute1", "subRoute2",
subroute3")
```

Approach 2: Define a custom thread pool in *camelContext.xml*. You can then reference the custom thread pool using the *executorServiceRef* attribute to look up the thread pool by ID.

```
<threadPool id="customThreadPool"
threadName="customThread"
poolSize="300" maxPoolSize="300"/>

from ("mainRoute")
.multicast()
.aggregationStrategy (new
ResponseAggregator())
.parallelProcessing()
.executorServiceRef
(customThreadPool)
.to ("subRoute1", "subRoute2",
subroute3")
```

Approach 2 is preferred since the thread pool configuration goes into the configuration file (*camelContext.xml*) and can be modified without changing any code.

A few points about the configuration for the custom thread pool: Define a unique pool for each parallel processing flow and tune the pool based on the requirements of the route. A few factors to consider – the maximum load expected to be handled by the main route, the number of sub-routes to be processed by the thread pool, and the ratio of processing time vs wait time expected when the threads are executing the sub-routes. Determine the number of threads needed and use the same value for both *poolSize* and *maxPoolSize*. Quoting from Javadoc on the behaviour of pool sizes: “If there are more than *corePoolSize* but less than *maximumPoolSize* threads running, a new thread will be created only if the queue is full.”

b. Streaming

When using the parallel processing EIPs, we specify an *AggregationStrategy* that aggregates/combines the responses from parallel sub-routes into one combined response. However, by default, the responses are aggregated in the same order in which the parallel sub-routes are invoked. This default behaviour causes aggregation tasks to spend lot of CPU in the polling mechanism. By enabling streaming we can reduce this CPU usage and provide a better performance. With streaming, the responses will be processed as and when they are received rather than in the order of multicast route invocation.

 **Note:** Streaming should be applied only if the *AggregationStrategy* does not depend on the order of responses from the sub-routes. An example is:

```
from ("mainRoute")
.multicast()
.aggregationStrategy (new
ResponseAggregator())
.parallelProcessing()
.streaming()
.executorService(executorService)
.to ("subRoute1", "subRoute2",
subroute3")
```

c. Parallel aggregation

AggregationStrategy combines the responses from parallel sub-routes into one final response. This is done by invoking the *aggregate()* method in the *AggregationStrategy* class for each response received from the sub-route.

By default, Camel synchronises the call to the aggregate method. If parallel aggregation is enabled, then the aggregate method on *AggregationStrategy* can be called concurrently. This can be used to achieve higher performance when the *AggregationStrategy* is implemented as thread safe.

 **Note:** Enabling parallel aggregation would require the *AggregationStrategy* to be implemented as thread safe. An example is:

```
from ("mainRoute")
.multicast()
.aggregationStrategy(new
ResponseAggregator())
.parallelAggregate()
.parallelProcessing()
.streaming()
.executorService (executorService)
.to ("subRoute1", "subRoute2",
subroute3")
```

END 

Disclaimer

The views expressed in this paper solely belong to the author, and does not reflect the views of their employer (Sabre).

References

- [1] <https://camel.apache.org/components/latest/eips/enterprise-integration-patterns.html>
- [2] <https://camel.apache.org/components/latest/eips/multicast-eip.html>
- [3] <https://camel.apache.org/manual/latest/threading-model.html>
- [4] https://access.redhat.com/documentation/en-us/red_hat_fuse/7.3/html/apache_camel_development_guide/basicprinciples#BasicPrinciples-Thread
- [5] https://access.redhat.com/documentation/en-us/red_hat_fuse/7.3/html/apache_camel_development_guide/msgrount#MsgRout-Multicast

By: Lokesh Chenta

The author is a principal software architect in the lodging, ground and sea (LGS) team at Sabre's Bengaluru Global Capability Center. He has more than 14 years of experience in designing and developing scalable, enterprise applications.